SCHOOL CLINIC MANAGEMENT SYSTEM

Software Development Document

An Object-Oriented Programming Project
Presented to the Faculty of the
College of Computer Studies
University of the Immaculate Conception
Father Selga St., Davao City, Philippines

Team Members: Author names were not supplied in the available project materials.

OCTOBER 2026
Documentation finalized: October 25, 2026

Prepared from the current source tree, supplied sample template, and executed checks. Institutional wording follows the template; this document does not assert instructor approval.

Document Guide and Verification Basis

This final document follows the supplied Software Development Document template: Software Purpose; Objectives, Features, and Modules; Technology Used; User Manual; Technical Manual; Project Development Phases; Limitations and Future Development; Appendix; and Comments/Grading. The requested Test Cases section is the final section. Template examples about grading software and unrelated social applications have been replaced with SCMS content.

The evidence baseline is the workspace on October 25, 2026. Source code takes precedence over older README statements and prior conversational claims. The original PDF is preserved. Actual team names, a signed project schedule, instructor identity, rubric scores, and a submission deadline were not supplied; their absence is stated explicitly.

Evidence	Observed result
tools/verify.cmd	Executed successfully on October 25, 2026: 56 domain/security checks; 36 UI checks; recent-visit and 10,004-record search checks; 49 persistence checks.
tools/verify-desktop.cmd	Executed successfully on October 25, 2026: 19 native Swing workflow checks.
Source inspection	GUI routing, domain models/services, schema/repositories, validation, authentication, AI client, launchers, and test assertions inspected.
AI service evidence	Live success remains unverified. Earlier user-provided runtime screenshots showed missing configuration, HTTP 404, and timeout failures. No cloud call was made for this documentation.
Screenshots	Included screenshots are produced by the project verification fixtures. They are demonstration/test data, not real student records.

“Pass” in the final table means the stated scenario was exercised by a successful check during this documentation run. “Not Tested” covers unexecuted scenarios. Successful regression suites do not prove complete production readiness.

1. Software Purpose

Project Description

The School Clinic Management System (SCMS) is a Java desktop application for recording school-clinic encounters, registering student and employee patients, tracking medicine inventory and dispensing, generating visit reports, and preparing guardian notification messages. It separates administrative account management from daily nursing operations. Normal startup stores information in a local encrypted H2 database. An explicit demo mode uses session-only fixtures.

Background and Problem Statement

The implementation addresses the need to organize patient identification, visit documentation, allergy checks, stock balances, and follow-up messages within one role-aware workspace. Its rationale is to reduce fragmented record handling and make the relationships between patients, staff, medicine transactions, and notifications explicit. No survey, institutional deployment statistics, or measured efficiency improvement was supplied, so no quantitative benefit is claimed.

Purpose and Key Value

• For school nurses: searchable patient and consultation records, stock visibility, and controlled medicine recording in a consistent workspace.

• For administrators: authenticated staff-account creation, role and activation management, a campus catalog, and an audit viewer.

• For record integrity: duplicate checks, foreign keys, transaction rollback, immutable visit records, and permissions enforced through services.

• For maintainers and evaluators: a concrete object-oriented structure with shared interfaces, subtype behavior, reusable Swing components, and executable checks.

Target Users and Scope

Target users are school nurses, nurse assistants, doctors, and administrators. Patients are students and employees; patients and guardians do not have self-service login screens. The system is a single-process desktop application using one local database instance. The campus catalog is maintained independently: user and patient campus membership is not defined in the schema.

2. Objectives, Features, and Modules

2.1 Core Objectives

• Provide authenticated, role-specific access to clinic and administrative screens.

• Record patient registration, allergies, consultations, and medicine transactions using established model fields.

• Maintain unexpired medicine balances and retain a reason for warnings overridden through the nurse UI.

• Persist accounts, clinical records, stock, notification status, and audit events across restarts.

• Demonstrate encapsulation, inheritance, polymorphism, abstraction, and separation of interface, domain logic, and persistence.

2.2 Key Features and Implementation Status

Feature	Current behavior and status
Authentication	Implemented: first-administrator setup, salted password hashing, active-account checks, lockout, logout, and role-specific launchers.
Patients	Implemented: register Student or Employee, search, view details, and add allergy. General profile editing/deletion is not exposed.
Consultations	Implemented: registered patient lookup, symptom choice, reason, optional remarks, staff-bound save, history/search. Visits are immutable.
Medicine inventory	Implemented: add batch, restock, quantity/expiry validation, stock totals, low-stock and expiry summaries, forecast display.
Dispensing	Implemented: positive quantity, patient/medicine validation, allergy warning, cancellation, documented override, earliest-expiry stock consumption.
Notifications	Implemented: transaction-linked draft, preview, and manual delivery confirmation. No SMS/email transmission.
Reports	Implemented: format the selected stored consultation as text in a dialog. No built-in PDF/Excel export claimed.
Superadmin	Implemented: account creation/edit/deactivation, campus creation/edit, searchable audit log, actual system totals.
AI Nurse Assistant	Implemented client and UI. Optional external Gemini request; live successful response not verified. It receives typed text only and cannot execute clinic actions.

2.3 System Modules (OOP Structure)

```
clinic.app.Main -> GUI -> ApplicationContext -> Router
  app.nav: NurseShell / SuperadminShell / MainShell
  app.ui: auth, dashboard, patients, consultations, dispense,
          inventory, notifications, reports, settings, assistant
clinic.core: Login, PasswordHasher, ClinicSystem
clinic.model: Person -> Patient -> Student / Employee
              User -> Administrator / SchoolNurse /
                      NurseAssistant / Doctor
              MedicalRecord, MedicineTransaction, Medicine
clinic.service: SecuredClinicSystem, InMemoryClinicSystem,
                Inventory, SafetyChecker, PatternDetector,
                RestockForecast, AdminService, AuditLog
clinic.persistence: Database, Schema, Sql,
                    AccountRepository, ClinicRepository,
                    AdminRepository
clinic.ai: GeminiAiService, AiServiceException
```

Object-Oriented Principles

Encapsulation: patient allergies/history are held privately and returned through copies; medicine quantities change through validated operations. Inheritance: Patient extends Person; Student and Employee specialize Patient. User is a separate hierarchy, not a Person subclass. Polymorphism: role subclasses override canAccess; MedicalRecord and MedicineTransaction implement Formattable, whose toReport contract is used by the formatter. Abstraction: ClinicSystem specifies clinic operations, while secured services and repositories provide implementations. Composition: ApplicationContext supplies authentication, administration, storage, and session-bound clinic services to screens.

Role Permissions

Operation	Nurse	Assistant	Doctor	Admin
Register/view patient; add allergy	Yes	Yes	No	No
Record consultation	Yes	Yes	No	No
Read full visit history	Yes	Own visits via UI	No	No
Dispense / view stock	Yes	Yes	No	No
Add / restock medicine	Yes	No	No	No
Generate visit report	Yes	No	Yes	No
Users / campuses / audit	No	No	No	Yes
AI panel in navigation	Yes	No	No	No

The assistant own-visit list is an application query restriction; broader getVisitHistory is denied to that role. Doctor access is report-oriented. The AI panel checks SchoolNurse on construction; its standalone Gemini service does not enforce a session or role. This is a known security boundary gap, not a backend authorization guarantee.

3. Technology Used

Category	Verified technology
Language/runtime	Java source compiled with JDK 21.0.7 during current checks; Java 21 is the documented baseline.
GUI	Java Swing/AWT, SwingWorker, CardLayout, layout managers, reusable table/dialog components.
Database	H2 2.5.252 embedded JDBC; encrypted file storage with AES; schema migration to version 2.
Build	javac and java through CMD launchers plus inline PowerShell. No Maven or Gradle build is present.
Development tools	VS Code Java configuration is present; Git workspace is available. IDE use is not required to run.
Testing	Standalone Java assertion/check executables in tests/ and demo/. No JUnit dependency is used.
Cloud client	Built-in java.net.http.HttpClient, HTTPS, Gemini generateContent REST call. No provider SDK.
Runtime dependency	Pinned H2 JAR downloaded and SHA-256 checked by tools/dependencies.cmd.
Documentation generation	This PDF is generated using PDFKit; the sample is read with pdf-parse. These are documentation tools only and are not app runtime dependencies.

Operating Environment

The provided launchers target Windows CMD and PowerShell. Desktop UI checks were run in the available Windows environment. The application sets a minimum window size of 1100 x 700 and initially opens at 1280 x 800. Cross-platform packaging, alternate DPI, screen readers, and multiple-monitor behavior have not been fully validated. No benchmark establishes minimum RAM or processor requirements. A JDK, a writable project/data location, and the H2 dependency are required. Internet is needed for the first driver download and optional AI calls; local clinic operations use embedded storage.

4. User Manual

4.1 Installation & Startup

Place the complete project folder in a writable location. Install JDK 21 and verify that java and javac are on PATH. Open PowerShell in the SchoolClinicSystem folder. Run the normal launcher below; it verifies/downloads H2, compiles src into out, and opens the login window.

```
java -version
javac -version
.\Run.cmd
```

There is no required splash screen or public role-selection registration page. The entry point goes to shared authentication. A dedicated role launcher restricts the accepted role; it does not create accounts or bypass login.

4.2 First-Time Setup and Login

On an empty production database, use first-run administrator setup to enter an administrator ID, name, and password. Sign in with those credentials, open Users, and add the staff accounts with their appropriate roles. Sign out, then sign in as the nurse or other authorized staff member. Production does not install the sample credentials. Three unsuccessful attempts temporarily lock the account for 60 seconds. Inactive accounts cannot sign in.

For isolated demonstration only, run .\NurseDemo.cmd and use the sample Nurse ID N001 with password nurse123. This launcher recompiles and uses --demo. Demo data disappears at exit. Do not interpret a demo save as a production database write.

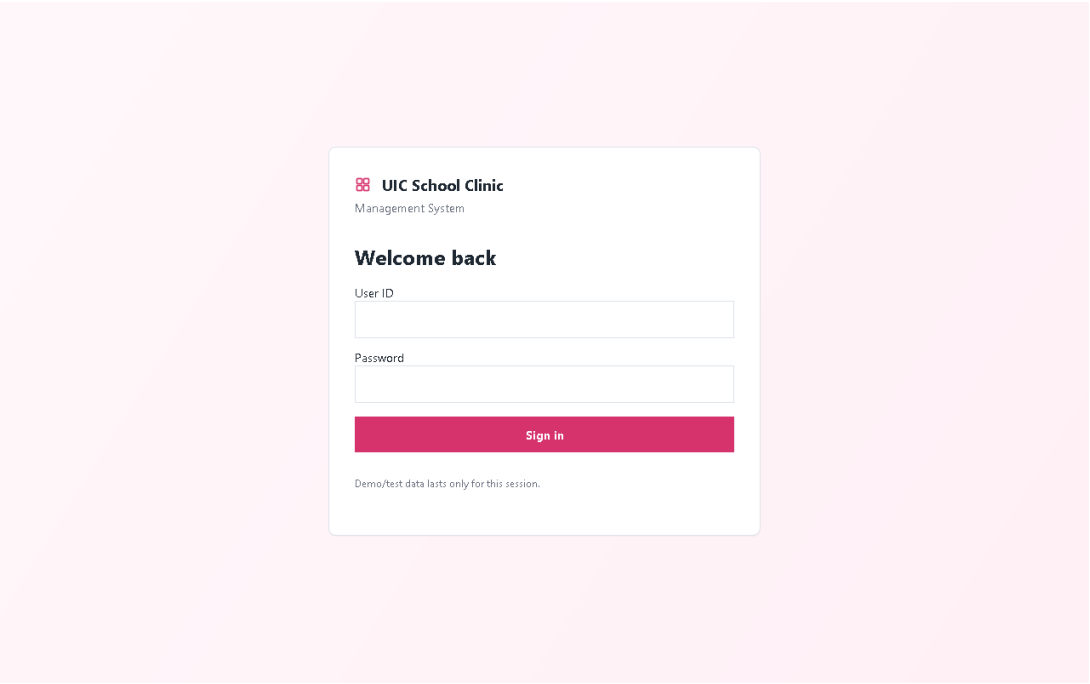


Figure 1. Shared login screen from the automated fixture; sample data only.

4.3 Nurse Dashboard and Navigation

After nurse login, the sidebar opens Nurse dashboard. Summary cards show registered patients, today's consultations, low-stock medicines, and stock expiring within 30 days. Recent Clinic Visits provides a name/ID search, Clear control, This week / Today / All scope, pagination, and New Consultation. This week begins Monday at midnight and ends at query time. Refresh requests actual service data. Sidebar selection identifies the active page. Revisited cached data pages reload their tables.

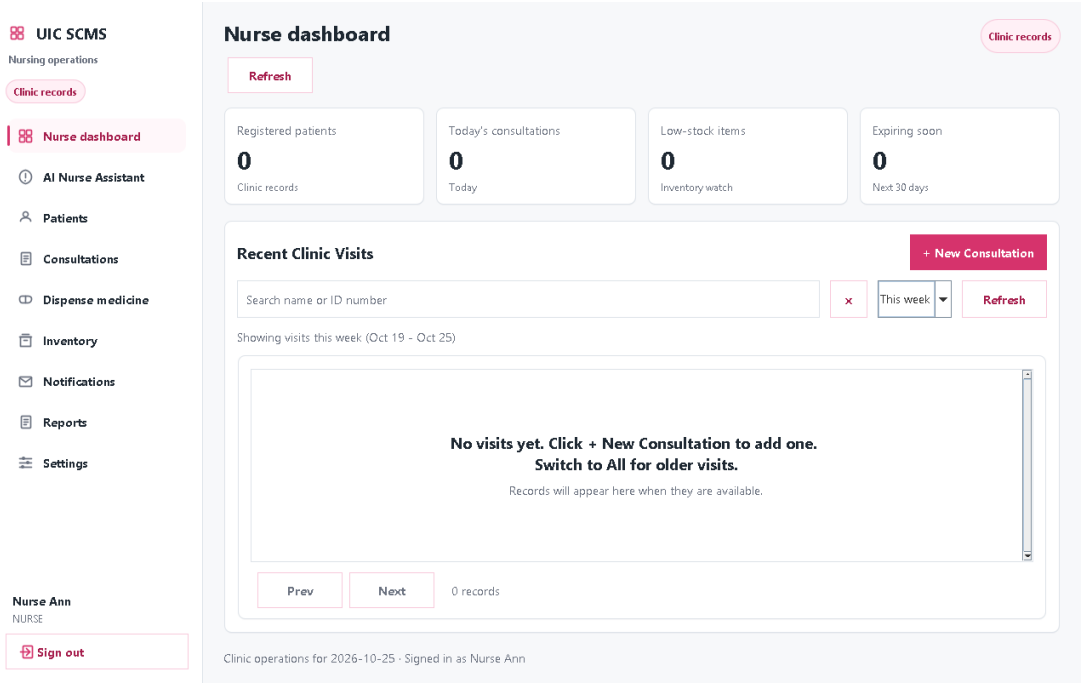


Figure 2. Nurse dashboard rendered by UI checks using an isolated fixture.

4.4 Register, Search, and Review a Patient

Open Patients. Search by name, ID, section, or position. Select Register patient and supply ID, name, contact, Student/Employee type, course/department, section/position, and a positive year level for students. Save submits through the clinic service; duplicate IDs and invalid values are rejected. After success, the table reloads. Select a row and use Details for profile information or Add allergy to record a nonblank allergy. There is no general Edit patient or Delete patient action.

4.5 Record a Consultation

Register the patient first. From the dashboard or Consultations page, open New Consultation / Record visit. Enter the existing Patient ID, choose a symptom, enter the reason, and optionally enter remarks. Save validates the patient and records the signed-in staff ID, then refreshes the relevant list. Blank symptom/reason or unknown patient prevents saving. The form offers one symptom choice even though the service model accepts a list. Consultations can be retrieved; editing a saved visit is not a supported UI action.

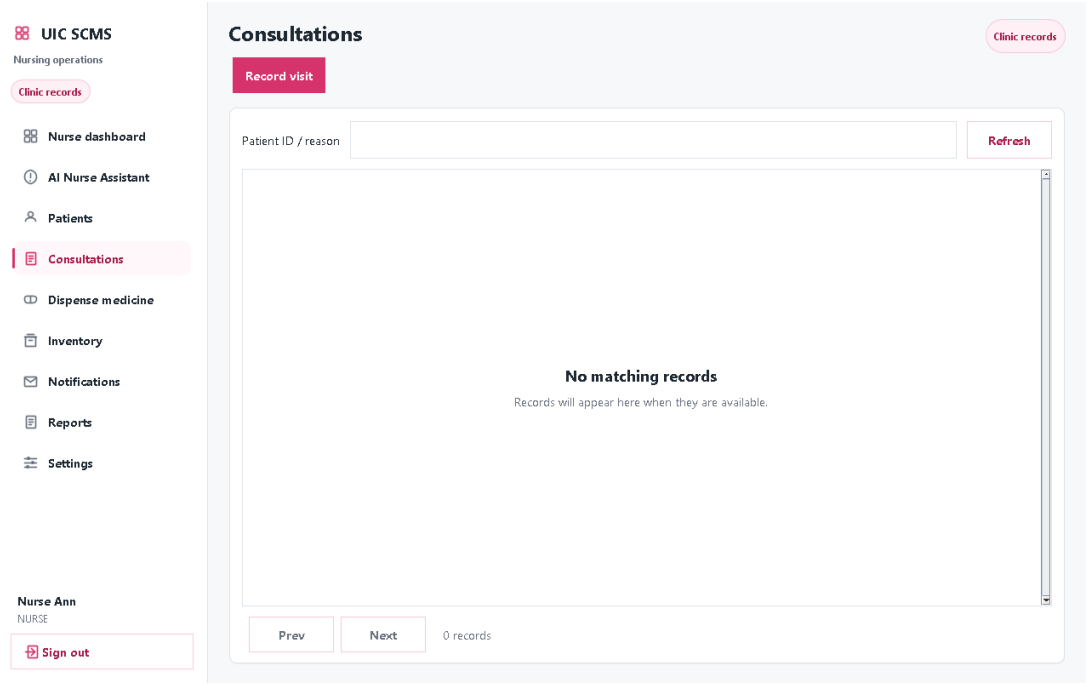


Figure 3. Consultation listing from UI verification.

4.6 Add and Restock Medicines

Open Inventory. Add medicine records a named batch with positive quantity, expiry in YYYY-MM-DD format, and optional remarks. Restock uses the selected medicine plus quantity and expiry. A past expiry or invalid quantity is rejected. Stock and forecast displays unexpired available stock and estimated days remaining; -1 means there is no usage history. Low-stock totals combine available batches for each medicine.

4.7 Dispense Medicine

Open Dispense medicine. Enter an existing Patient ID, medicine name, and positive whole quantity, then choose Check safety and dispense. Review any allergy warning. Cancel leaves stock unchanged. An override through the UI requires a meaningful reason of at least ten characters. A successful operation returns a transaction ID and reduces stock across available batches, earliest expiry first. Insufficient stock, invalid identity, or a database failure must not be reported as success. Retain the transaction ID if creating a guardian notification.

4.8 Guardian Notifications

Open Notifications and choose Create notification. Provide Patient ID, that patient's dispensing Transaction ID, and reason. Use View message to inspect the generated text. Deliver it outside the application. Choose Mark sent only after delivery and confirm the prompt. The status changes from PENDING to SENT; repeating the same sent update is rejected. The application does not actually send email or SMS.

4.9 Reports, Accounts, and Logout

In Reports, select a consultation and choose Generate selected report to view its formatted text. A doctor can access reports through the restricted clinic workspace. In nurse Settings, review signed-in ID, role, and storage location. Settings is informational; password change is not implemented. Use Sign out to clear the session and return to authentication. Closing the

application requires confirmation; cancel retains the window.

4.10 Superadmin Workflow

Sign in with an administrator account. Users provides account creation, role/name/active-status editing, and deactivation. The current administrator cannot modify their own account through this workflow. Campuses adds unique IDs and edits names. Audit log searches events and denied operations. Admin screens do not expose nurse dispensing. Campus entries do not assign users or patients to campuses.

4.11 AI Nurse Assistant

The AI panel is available in nurse navigation. Set GEMINI_API_KEY privately in the launching process environment and optionally GEMINI_MODEL before starting the app. Never place a real key in source or submission files. The current source default is gemini-3.1-flash-lite; model availability depends on the configured provider project. NurseDemo.cmd currently removes GEMINI_MODEL for its Java child process, so that launcher uses the source default.

Enter a general non-identifying question, confirm the privacy checkbox, and Send. Enter sends; the UI advertises Shift+Enter for a new line, whose exact key binding is not independently verified here. Send is disabled while a background request runs. Clear conversation removes local displayed messages when idle. Errors appear below the privacy control. The UI retains a visible conversation, but each request sends only the current question, not previous messages.

Do not enter student identifiers, contact details, credentials, or sensitive records. Basic patterns block obvious emails, phone numbers, and labeled IDs, but cannot guarantee de-identification. No patient records are automatically loaded into the prompt. The system instruction requests cautious support and draft labeling; it is not a clinical validation mechanism. Live successful operation remains unverified and prior failures are acknowledged.

5. Technical Manual

5.1 Development Environment Setup

Open the existing folder in VS Code with Java tooling, or use the Windows command line. Confirm java and javac resolve to a compatible JDK. No Maven import or SQLite configuration is needed. The launchers compile source recursively with UTF-8. Normal startup uses Main -> GUI; NurseGUI and SuperadminGUI call the shared launch method with required role constraints.

```
.\Run.cmd
.\NurseDemo.cmd
cmd /c tools\verify.cmd
cmd /c tools\verify-desktop.cmd
```

If compiling manually, include lib/* at runtime for production database startup. The manual command printed in older README text omits this runtime dependency; the corrected form is below. A direct java invocation does not rebuild out. verify.cmd builds out-check, so it does not refresh out.

```
$sources = @(Get-ChildItem src -Recurse -Filter *.java |
  ForEach-Object FullName)
javac -encoding UTF-8 -d out @sources
java -cp "out;lib/*" clinic.app.Main
```

Architecture and Component Interaction

ApplicationContext.production opens Database at the clinic.database system property or data/scms. AccountRepository supplies password hashes, roles, active flags, and lockout state. AdminRepository supplies campuses/audit storage. ClinicRepository supplies a ClinicData proxy using database transactions. A clinic call is wrapped by SecuredClinicSystem and a session-version proxy. The latter checks identity/version under the login lock on every retained-service call. Logout and failed reauthentication revoke earlier clinic references.

Frequently used count, patient lookup, and stock queries have direct SQL paths. Many other ClinicRepository calls load an aggregate, reuse InMemoryClinicSystem rules, and persist changed rows inside a transaction. This is durable persistence, but not fully SQL-paged retrieval: filtering/aggregation can materialize records. PageResult and PagedTablePanel bound displayed rows to 25. SwingWorker handles loading; generation checks reject stale results.

5.2 Database Schema

Schema.java creates the relational tables below and applies the version-2 account lockout migration. IDs are generally VARCHAR(500); timestamps use TIMESTAMP(9). The complete creation script is reproduced in Appendix C from source, not replaced with the template's student/course/grades example.

Table	Fields and relationships
schema_version	id PK (must equal 1), version; controls additive schema migration.

Table	Fields and relationships
accounts	**id PK, name, role, password_hash, active; v2 adds failed_attempts and locked_until. Roles: ADMIN/NURSE/ASSISTANT/DOCTOR.**
campuses	id PK, name. Independent catalog, no patient/account FK.
audit_entries	seq identity PK; event_time, user_id, user_name, role, action, details, allowed. No account FK.
patients	id PK; name, contact, patient_type, course_department, section_position, year_level. Student year > 0; employee year is NULL.
allergies	patient_id FK -> patients; allergy. Composite PK (patient_id, allergy).
patient_history	patient_id FK -> patients; ordinal, detail. Composite PK (patient_id, ordinal); no general history-edit screen.
visits	id PK; seq identity unique; patient_id FK, staff_id FK -> accounts; reason, assessment, medication, visit_time.
symptoms	visit_id FK -> visits; ordinal, symptom. Composite PK (visit_id, ordinal).
medicine_batches	ordinal PK; name, quantity >= 0, expiry, batch_date, remarks. Names need not be unique; multiple batches are permitted.
dispensing	id PK; seq identity unique; patient_id/staff_id FKs; medicine, quantity > 0, expiry, remarks, dispensed_at; unique(id, patient_id).
notifications	id PK; seq identity unique; patient_id FK, transaction_id, reason, created_at, status; composite FK to dispensing(id, patient_id).

The notification composite foreign key enforces ownership of the transaction by that patient. Medicine names in dispensing are text, not a foreign key to one batch, because one dispensing can consume multiple batches. The model stores the earliest consumed expiry in its transaction summary. Indexes exist on visits(patient_id, visit_time) and audit_entries(event_time).

Security, Validation, and Error Handling

Passwords use PBKDF2WithHmacSHA256 with a random 16-byte salt, 210,000 iterations, a 256-bit result, and constant-time comparison. Account activation and three-attempt/60-second lockout checks occur in Login. Login clears earlier authentication before trying new credentials. Administration requires an active administrator session. Clinic permissions are checked by the service rather than only by navigation. Staff IDs on visits/dispensing must match the signed-in identity.

Database uses an application lock for one process, local AES-encrypted H2 files, a random local key file, and owner-oriented filesystem permissions where supported. Prepared statements bind variable query values. Transaction failure rolls back before reporting a storage error. Audit details avoid passwords, patient names, allergy text, and free-text override reasons; the override reason remains in the clinical transaction. Audit permission

entries indicate that access was allowed, not necessarily that the later domain operation completed successfully.

FormPopup validates required values and saves asynchronously. Domain models check blanks and positive quantities; repositories add foreign-key/check constraints. Detached read objects prevent mutation of stored state through a read reference. The boolean legacy dispensing API still permits a confirmed override without the nurse form's ten-character text requirement; UI documentation must not imply that every legacy caller enforces that text rule.

AI Integration Boundaries

GeminiAiService uses HttpClient with a 10-second connection timeout and 35-second request timeout; HTTPS requests carry x-goog-api-key in a header. Input is limited to 3,500 characters and output requested at 1,024 tokens. There are no automatic retries. The model name is validated, and HTTP authentication, quota, timeout, 404, and network failures receive user messages. Error detail redacts the configured key. No key is reproduced in this document.

The JSON implementation is a custom string encoder/parser, not a full JSON library: it extracts the first text field and does not comprehensively validate candidate structure, finish reason, multipart content, or safety metadata. The panel does not recheck session identity for every AI request and has no complete cancellation/late-result lifecycle handling. These limitations require attention before production use. A shared privileged provider key should be kept on an authenticated server rather than distributed with a desktop app.

5.3 Project Setup, Maintenance, and Recovery

Run tools/dependencies.cmd to obtain the pinned H2 JAR when missing. Its checksum is verified on every launcher run. Production creates the local schema and encryption key when first opening the database. Do not run sample schema resets against a real record store. To use a different database base path, supply -Dclinic.database before the Java main class.

```
java -Dclinic.database=data/scms -cp "out;lib/*" clinic.app.Main
```

For backup, close the application and copy the complete data folder, including the matching .mv.db and .key files, to a protected location. Keep these together; a missing key prevents opening an existing encrypted database. No automatic backup or tested recovery tool is included. A second process opening the same store is rejected. To update code, close the window and run the compilation launcher. Keep demo/test databases separate from real clinic data.

6. Project Development Phases

The sample asks for a weekly progress snapshot. The supplied materials do not establish an actual week-by-week schedule or individual team contributions. The evidence-supported implementation phases below describe the present artifacts, not invented dates or a retrospective time sheet.

Phase	Current evidence and deliverable
Requirements and modeling	ClinicSystem contract, Person/Patient and User hierarchies, permission constants, model classes, and source comments. Original stakeholder acceptance dates unavailable.
Domain implementation	Patient registration, immutable visits, inventory/batch operations, allergy checks, transaction reporting, notifications, and analytics services.
Desktop interface	Shared Theme and reusable forms/tables, role-specific shells, dashboard, nurse modules, and administration panels.
Persistence and security	H2 repositories/schema, encrypted storage, lockout persistence, session proxy, prepared queries, and rollback checks.
Optional AI integration	Gemini cloud client and nurse chat panel. Source present; live generation success unresolved in available evidence.
Verification and documentation	Current automated domain, UI, persistence, native workflow, and visit-search checks executed; final PDF produced on October 25, 2026.

No statement is made that the project followed the sample's 13-week schedule. Project management sign-off, instructor acceptance, and actual deployment history remain undocumented.

7. Limitations and Future Development

7.1 Current Limitations

• One application process owns the local database. The system is not a shared multi-workstation clinic server.

• No appointments, patient deletion/general profile editing, visit editing, password reset/change, or patient/guardian portal is exposed. Existing user and campus editing must not be confused with patient CRUD.

• Guardian messages require external manual delivery. Reports display text; export and automatic distribution are absent.

• Campus catalog entries have no enforced membership relationship with patients or accounts.

• SafetyChecker contains limited allergy matching, not a comprehensive drug interaction database. Forecast and outbreak rules are application heuristics, not diagnostic conclusions.

• Many persistent queries load aggregates into memory. The UI is paged but the persistence layer is not fully optimized for high-volume server workloads.

• AI live generation has not been established as working. Custom JSON handling, session authorization, bounded chat retention, cancellation, error wrapping, and privacy screening remain incomplete. Pattern screening cannot detect all sensitive information.

• Automatic backup, restore validation, retention policy, audit tamper resistance, institutional privacy compliance, and deployment hardening are not established.

7.2 Future Development Plan

The following are recommendations arising from the audit, not implemented or approved commitments. Prioritize reliable AI model discovery/configuration, a real JSON parser, per-request nurse-session checks, cancellable requests, and bounded chat history. Strengthen usable error wrapping and keyboard/accessibility verification. Add an authenticated server-side AI proxy before distributing shared credentials.

For storage and operations, introduce a tested backup/restore workflow, SQL-based filtering/pagination for larger datasets, and operational recovery guidance. New appointment, patient-edit, export, campus membership, or messaging capabilities require defined requirements, access rules, and migrations before implementation. Any expansion of medicine safety rules requires authoritative clinical input.

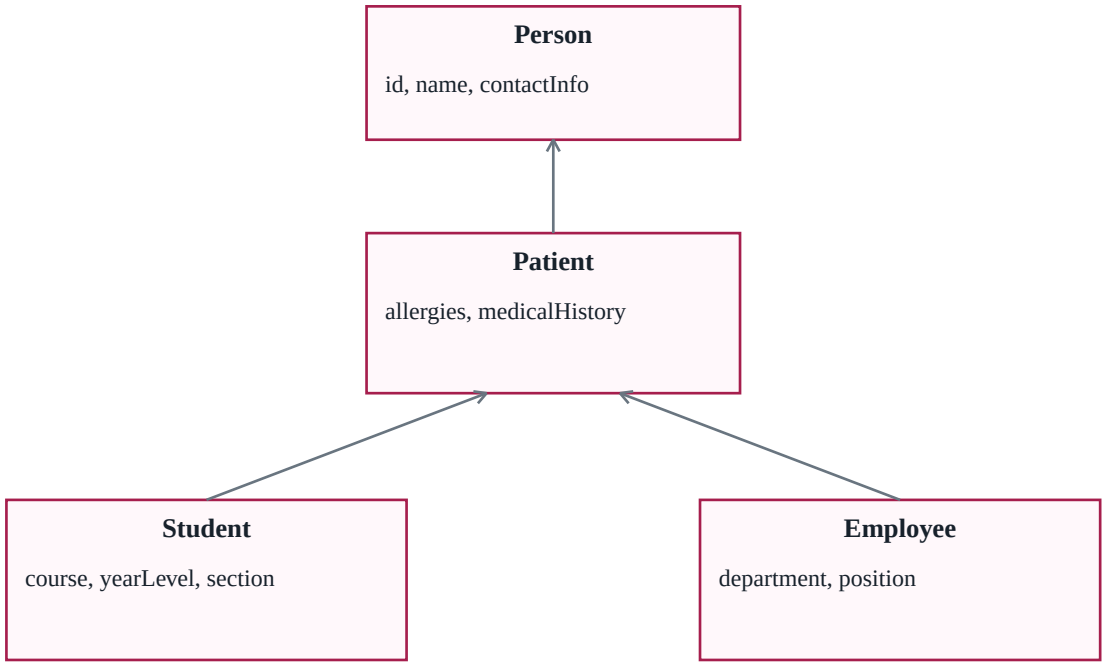
Technical Debt and Refactoring

Retained legacy NurseDashboard and ModulePage classes are not the routed operational panels. Documentation and comments occasionally lag source changes: for example, the current Gemini default differs from older README examples. Keep tests, launchers, and documentation aligned. Preserve the public clinic contract when replacing aggregate loading with more targeted repositories.

Appendix

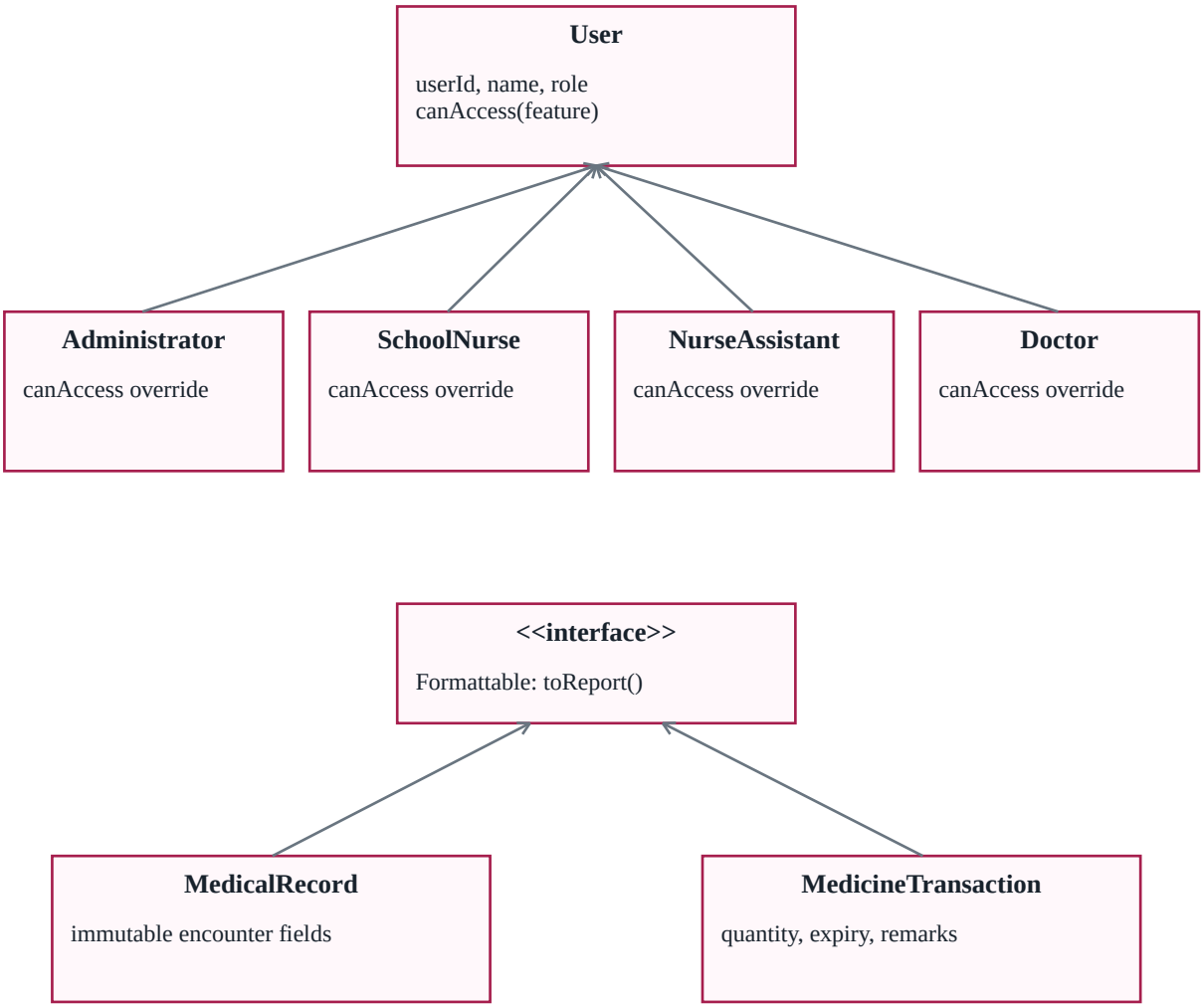
A. UML Class Diagram

The diagram below reflects actual inheritance and interfaces. User does not inherit Person. Dashed conceptual call relationships are explained in text rather than used to imply inheritance.



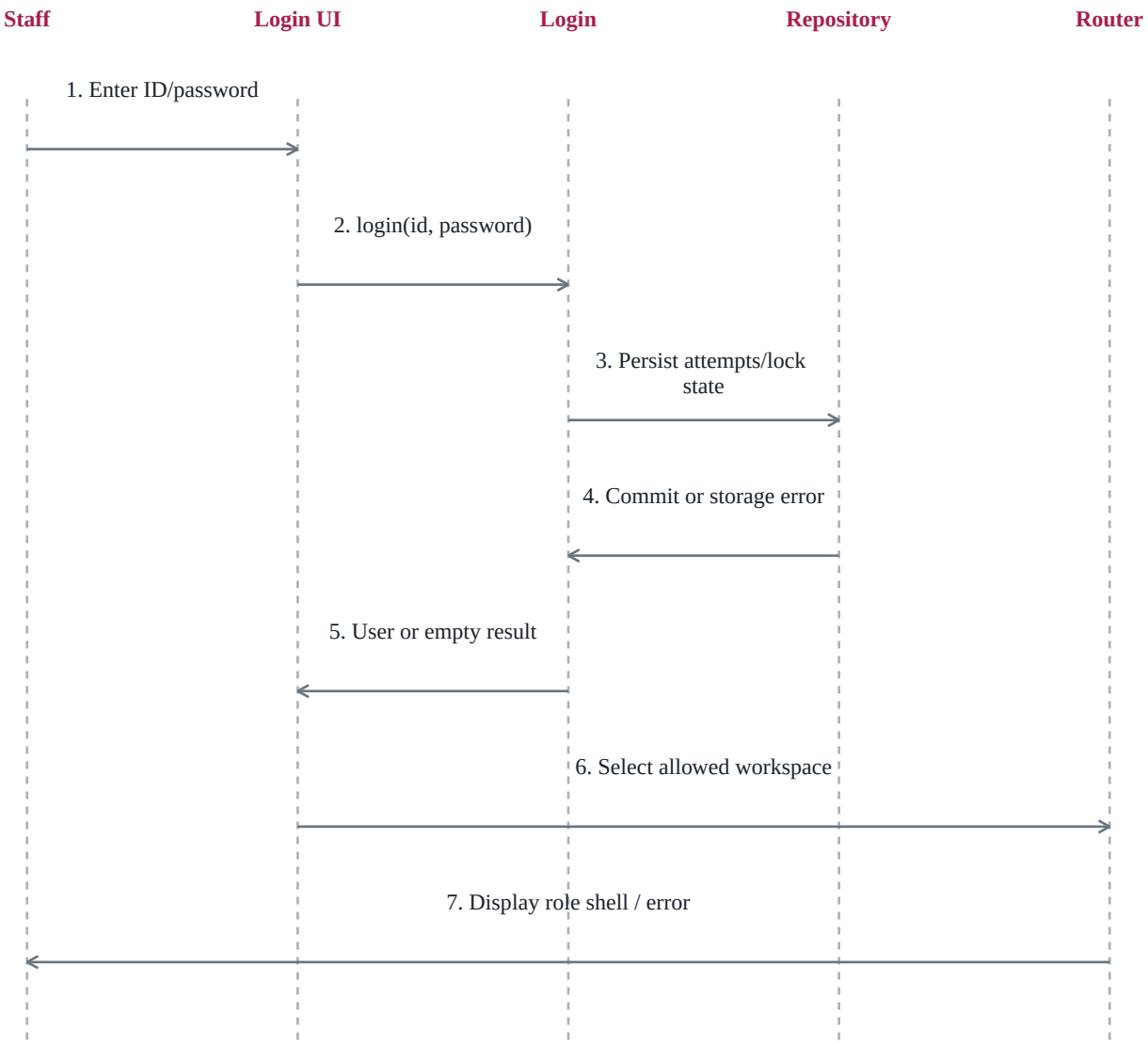
Inheritance arrows point toward the superclass. Patient.copy returns detached subtype-preserving profiles.

A. UML Class Diagram (continued)



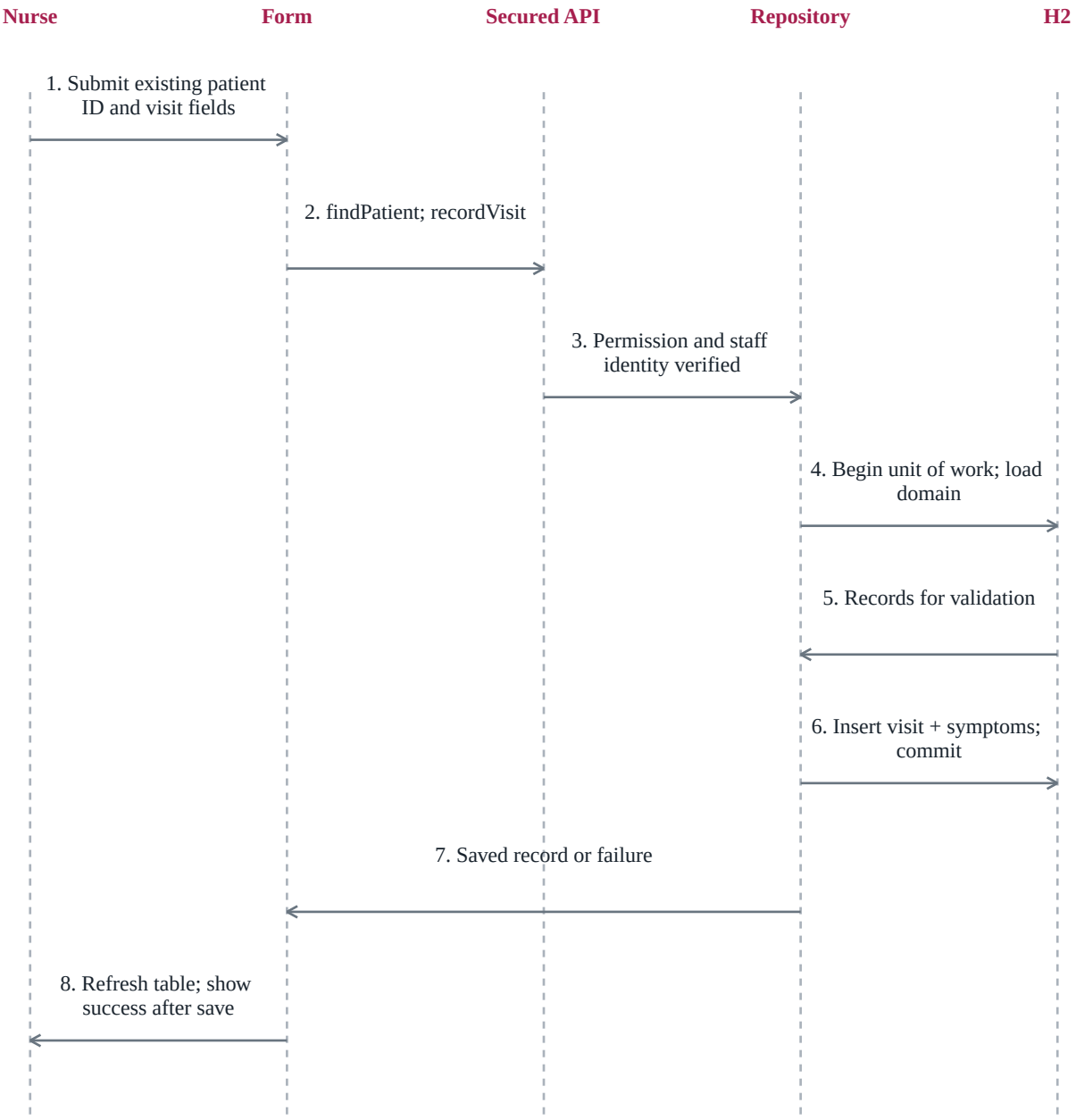
ClinicSystem is implemented by InMemoryClinicSystem and SecuredClinicSystem. The secured decorator delegates after checking role permissions; ApplicationContext supplies the session-bound proxy. Database, repositories, and Swing panels are composition dependencies, not subclasses of these model types.

B.1 Sequence Diagram - Login and Routing



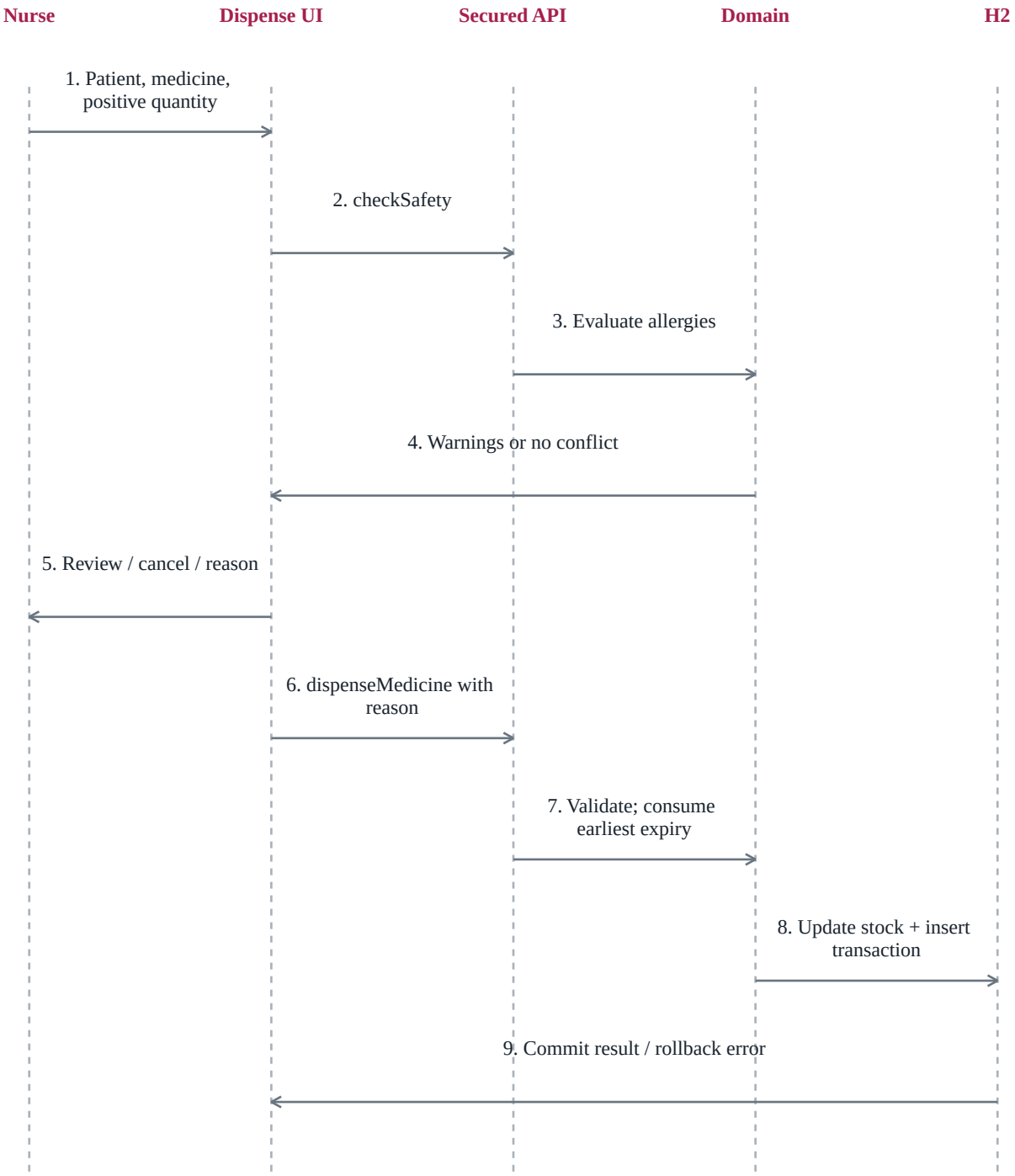
Login verifies password/active/lock state from its account cache loaded from AccountRepository. Password verification occurs locally; the repository step records attempts. A failed attempt clears any earlier session.

B.2 Sequence Diagram - Record Consultation



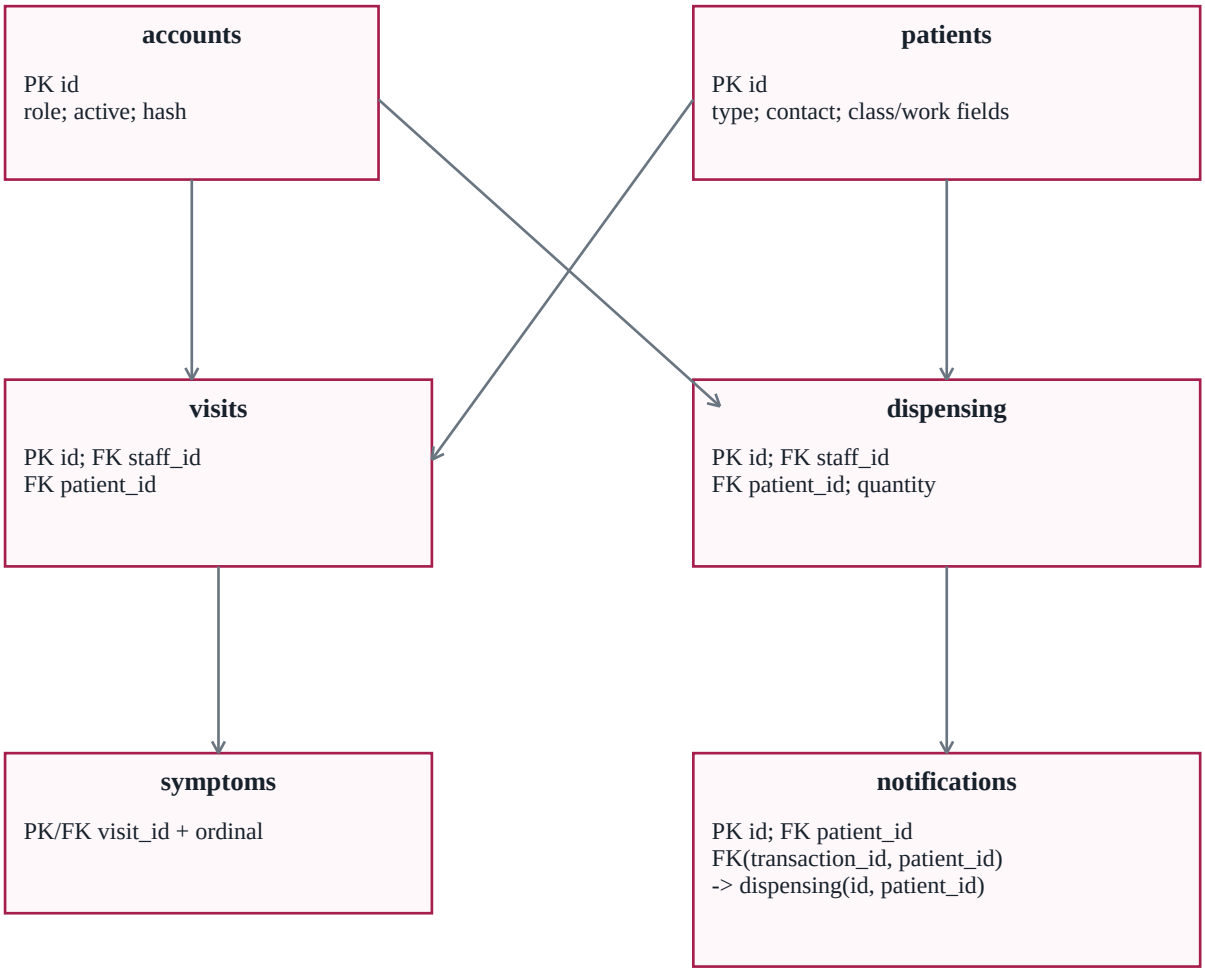
The service rejects unknown patient IDs and invalid symptoms. Foreign keys reject missing staff; a failed write rolls back rather than partially storing the encounter.

B.3 Sequence Diagram - Dispense Medicine



Safety inspection and dispensing are separate calls; dispensing revalidates the domain state in its own transaction. Cancel does not submit a stock mutation. The database transaction is supplied through ClinicRepository around the domain call.

C. Database Relationship Diagram



Arrows point from parent records to dependent records (one-to-many). Patients also have one-to-many allergies and patient_history entries. Campuses, schema_version, audit_entries, and medicine_batches are independent tables in the current schema. There is no foreign key from dispensing.medicine to an individual medicine batch.

C.1 Exact SQL Creation Script

The following DDL is extracted verbatim from Schema.java. The account migration statements following it create the current version-2 lockout fields. Do not execute against a production store as a reset script.

```
CREATE TABLE IF NOT EXISTS schema_version (
  id INT PRIMARY KEY CHECK(id=1), version INT NOT NULL
);
```

```
Đ
CREATE TABLE IF NOT EXISTS accounts (Đ
  id VARCHAR(500) PRIMARY KEY, name VARCHAR(500) NOT NULL,Đ
  role VARCHAR(20) NOT NULL CHECK(role IN ('ADMIN','NURSE','ASSISTANT','DOCTOR')),Đ
  password_hash VARCHAR(200) NOT NULL, active BOOLEAN NOT NULL);Đ
CREATE TABLE IF NOT EXISTS campuses (id VARCHAR(500) PRIMARY KEY, name VARCHAR(500) NOT
NULL);Đ
CREATE TABLE IF NOT EXISTS audit_entries (Đ
```

```
  seq BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY, event_time TIMESTAMP(9) NOT NULL,Đ
  user_id VARCHAR(500) NOT NULL, user_name VARCHAR(500) NOT NULL, role VARCHAR(20) NOT
NULL,Đ
  action VARCHAR(100) NOT NULL, details VARCHAR(500) NOT NULL, allowed BOOLEAN NOT
NULL);Đ
CREATE TABLE IF NOT EXISTS patients (Đ
  id VARCHAR(500) PRIMARY KEY, name VARCHAR(500) NOT NULL, contact VARCHAR(500) NOT
NULL,Đ
  patient_type VARCHAR(20) NOT NULL CHECK(patient_type IN ('STUDENT','EMPLOYEE')),Đ
  course_department VARCHAR(500) NOT NULL, section_position VARCHAR(500) NOT NULL,Đ
  year_level INT, CHECK((patient_type='STUDENT' AND year_level>0) OR
(patient_type='EMPLOYEE' AND year_level IS NULL)));Đ
CREATE TABLE IF NOT EXISTS allergies (Đ
  patient_id VARCHAR(500) REFERENCES patients(id), allergy VARCHAR(500) NOT NULL,
PRIMARY KEY(patient_id,allergy));Đ
CREATE TABLE IF NOT EXISTS patient_history (Đ
  patient_id VARCHAR(500) REFERENCES patients(id), ordinal INT NOT NULL, detail
VARCHAR(500) NOT NULL, PRIMARY KEY(patient_id,ordinal));Đ
CREATE TABLE IF NOT EXISTS visits (Đ
  seq BIGINT GENERATED ALWAYS AS IDENTITY UNIQUE, id VARCHAR(500) PRIMARY KEY,Đ
  patient_id VARCHAR(500) NOT NULL REFERENCES patients(id), reason VARCHAR(500) NOT
NULL,Đ
  assessment CLOB NOT NULL, medication CLOB NOT NULL, staff_id VARCHAR(500) NOT NULL
REFERENCES accounts(id), visit_time TIMESTAMP(9) NOT NULL);Đ
CREATE TABLE IF NOT EXISTS symptoms (Đ
  visit_id VARCHAR(500) REFERENCES visits(id), ordinal INT NOT NULL, symptom
VARCHAR(500) NOT NULL, PRIMARY KEY(visit_id,ordinal));Đ
CREATE TABLE IF NOT EXISTS medicine_batches (Đ
  ordinal INT PRIMARY KEY, name VARCHAR(500) NOT NULL, quantity INT NOT NULL
CHECK(quantity>=0),Đ
  expiry DATE NOT NULL, batch_date DATE NOT NULL, remarks CLOB NOT NULL);Đ
CREATE TABLE IF NOT EXISTS dispensing (Đ
  seq BIGINT GENERATED ALWAYS AS IDENTITY UNIQUE, id VARCHAR(500) PRIMARY KEY,Đ
  medicine VARCHAR(500) NOT NULL, patient_id VARCHAR(500) NOT NULL REFERENCES
patients(id),Đ
  staff_id VARCHAR(500) NOT NULL REFERENCES accounts(id), quantity INT NOT NULL
CHECK(quantity>0),Đ
  expiry DATE NOT NULL, remarks CLOB NOT NULL, dispensed_at TIMESTAMP(9) NOT NULL,Đ
  UNIQUE(id, patient_id));Đ
CREATE TABLE IF NOT EXISTS notifications (Đ
  seq BIGINT GENERATED ALWAYS AS IDENTITY UNIQUE, id VARCHAR(500) PRIMARY KEY,Đ
  patient_id VARCHAR(500) NOT NULL REFERENCES patients(id), transaction_id VARCHAR(500)
NOT NULL,Đ
  reason VARCHAR(500) NOT NULL, created_at TIMESTAMP(9) NOT NULL,Đ
  status VARCHAR(20) NOT NULL CHECK(status IN ('PENDING','SENT')),Đ
  FOREIGN KEY(transaction_id,patient_id) REFERENCES dispensing(id,patient_id));Đ
CREATE INDEX IF NOT EXISTS visit_patient_time ON visits(patient_id,visit_time);Đ
CREATE INDEX IF NOT EXISTS audit_time ON audit_entries(event_time);Đ

ALTER TABLE accounts ADD COLUMN IF NOT EXISTS failed_attempts
  INT NOT NULL DEFAULT 0 CHECK(failed_attempts>=0);
ALTER TABLE accounts ADD COLUMN IF NOT EXISTS locked_until
  BIGINT NOT NULL DEFAULT 0;
```

D. Test Cases Summary and Evidence Map

The comprehensive four-column Test Cases table appears at the very end, as requested. Current execution results: 56 domain/security checks, 36 UI checks, 49 persistence checks, 19 desktop workflow checks, and additional visit/search assertions. No failure was reported by these executed suites. A live successful AI response is not established.

Check source	Scope

Check source	Scope
tests/DomainChecks.java	**Login, permissions, stock integrity, allergy overrides, notifications, detached reads, session revocation.**
tests/UiChecks.java	Paged loading, stale/error recovery, role navigation, AI-page opening, fixture screenshots.
tests/PersistenceChecks.java	Restart preservation, FK constraints, SQL-shaped literal search, rollback, key/account persistence, migration.
tests/DesktopChecks.java	19 native dialog/navigation workflows using Swing actions and isolated fixture data.
demo/ DashboardConsultationChecks.java; demo/VisitSearchDemo.java	Recent-visit ordering, role restrictions, date/name/ID filtering, 10,004-visit data set.

E. Source References

Primary sources: src/clinic/app/{GUI,Router,ApplicationContext}.java; app/nav/{NurseShell,MainShell}.java; app/ui/{patients,consultations,dispense,inventory,notifications,reports,settings,assistant}; core/{Login,PasswordHasher,ClinicSystem}.java; model/{Person,Patient,Student,Employee,User,SchoolNurse,NurseAssistant,Doctor,Administrator,Formattable,MedicalRecord,MedicineTransaction}.java; service/{SecuredClinicSystem,InMemoryClinicSystem,Inventory,SafetyChecker,PatternDetector,RestockForecast}.java; persistence/{Schema,Database,ClinicRepository,AccountRepository,AdminRepository,Sql}.java; ai/GeminiAiService.java; tools/verify.cmd; tools/verify-desktop.cmd.

Template reference: OOP Final Project Document Sample (1).docx.pdf supplied from Downloads. It is used as a structural and typographic guide. Sample names, features, diagrams, and example results are not evidence about SCMS. No exposed API credentials or real patient records have been included.

Comments/Grading

This section preserves the template's evaluator-facing purpose. Scores and signatures must be assigned by the instructor; no grading has been provided in the evidence. The sample does not specify category maximums, so no weights are invented.

Criteria/Section	Maximum Score	Actual Score
Software Purpose	Not specified	Not assessed
Objective & Feature Accomplished	Not specified	Not assessed
Technology Stack	Not specified	Not assessed
User Manual	Not specified	Not assessed
Code Quality	Not specified	Not assessed
UML Diagram	Not specified	Not assessed
Database Structure & ERD	Not specified	Not assessed
Total	100 (sample total)	Not assessed

Instructor Feedback: Not supplied; reserved for instructor evaluation.

Final Grade: Not assigned / 100.

Instructor Signature: Not supplied.

Evaluation Date: Not supplied.

Test Cases

Executed on October 25, 2026 using isolated fixtures or temporary databases. D = DomainChecks; U = UiChecks; P = PersistenceChecks; N = DesktopChecks; V = DashboardConsultationChecks / VisitSearchDemo. IDs and source labels are included inside Test Case so the table retains exactly the four required columns. “Not Tested” does not imply success.

Feature	Test Case	Expected Outcome	Pass/Fail
Setup	TC01 D/P/N: Bootstrap an empty store with an administrator.	Administrator can sign in; setup is not offered again after restart.	Pass
Authentication	TC02 D/N: Submit incorrect credentials.	Login fails; no privileged session remains.	Pass
Lockout	TC03 D: Make three failed attempts, then use the correct password before 60 seconds.	Correct password is temporarily rejected; succeeds after simulated expiry.	Pass
Persistent lockout	TC04 P: Restart after three failed nurse attempts.	Lockout remains; another account can still sign in.	Pass
Account activation	TC05 D/P: Deactivate a nurse/doctor and attempt login.	Inactive account is rejected, including after restart.	Pass
Account update	TC06 D: Change a role and reactivate while retaining the password.	New role takes effect; existing password still authenticates.	Pass
Account creation	TC07 N: Administrator adds a staff user through the form.	User is created and the table refreshes.	Pass
Unauthorized account action	TC08 D: Add user without administrator session.	SecurityException; no authorized account creation.	Pass
Campus catalog	TC09 D/P/N: Add then rename a campus.	Updated name displays and survives restart.	Pass
Campus duplicate	TC10 D/P: Add an existing campus ID as new.	Duplicate is rejected.	Pass
Nurse authorization	TC11 P/U: Nurse accesses users/ audit or opens Superadmin shell.	Access denied; nurse navigation omits administration.	Pass
Role launcher	TC12 U: Administrator opens nurse-only route; nurse opens admin-only route.	Role mismatch rejected.	Pass

Feature	Test Case	Expected Outcome	Pass/Fail
Logout	**TC13 D/P: Retain a clinic reference, log out, then call it.**	**SecurityException; retained service is revoked.**	**Pass**
Relogin	TC14 D: Reuse old reference after same-user login.	Old session version is still rejected.	Pass
Patients	TC15 N: Submit required registration fields.	Patient is registered and visible after refresh.	Pass
Required fields	TC16 N: Submit an incomplete patient form.	Registration is prevented.	Pass
Cancel form	TC17 N: Cancel registration.	Patient data remains unchanged.	Pass
Patient duplicate	TC18 D/P: Register an existing patient ID.	Duplicate rejected; no duplicate row.	Pass
Detached profiles	TC19 D: Mutate an allergy on a returned patient.	Stored profile is unchanged.	Pass
Allergy persistence	TC20 P: Add Penicillin allergy, close and reopen.	Stored allergy is present after restart.	Pass
Literal search	TC21 P: Search an apostrophe-containing name and SQL-shaped text.	Name matches literally; SQL-shaped text does not return all patients.	Pass
Consultation	TC22 D/N: Save a valid registered-patient consultation.	Visit is stored; form reports success and list refreshes.	Pass
Symptom validation	TC23 D: Record a visit with blank symptom.	Domain rejects it; history count unchanged.	Pass
Visit persistence	TC24 P: Restart after storing visit ID, timestamp, symptoms and remarks.	All checked fields retain their saved values.	Pass
Visit foreign key	TC25 P: Store a visit with nonexistent staff ID at repository boundary.	Database rejects it and no partial visit remains.	Pass
Staff identity	TC26 D: Dispense using another staff ID.	Service rejects impersonation.	Pass
Assistant history	TC27 P/V: Assistant queries visits recorded by nurse.	Other staff visits are excluded from assistant list.	Pass
Recent visits	TC28 V: Query recent visits and page sizes 25 and 15.	Newest first; each page respects requested limit.	Pass

Feature	Test Case	Expected Outcome	Pass/Fail
Visit filters	**TC29 V: Search name/ID with This week, Today, All and no-match query.**	**Filtering occurs before paging; scope results match fixtures.**	**Pass**
Date boundaries	TC30 V: Include Monday midnight and future-dated fixtures.	Monday is included in week; future visits excluded.	Pass
Search privacy	TC31 V: Inspect audit after patient-name search.	Audit records search action without the query text.	Pass
Inventory	TC32 N: Restock selected medicine with valid quantity/expiry.	New batch appears and total stock increases.	Pass
Expiry validation	TC33 D: Add a batch expiring yesterday.	Expired new batch rejected.	Pass
Low stock	TC34 D: Aggregate multiple batches of same medicine.	One medicine summary with combined available quantity.	Pass
Expiry summary	TC35 D: Scan expiry alerts with emptied batch present.	Empty batches excluded.	Pass
Safety warning	TC36 D: Check Amoxicillin for Penicillin-allergic patient.	Warning is returned.	Pass
Safety enforcement	TC37 D/P: Dispense warning medicine without confirmation.	Rejected; stored stock remains unchanged.	Pass
Cancel warning	TC38 N: Cancel safety warning dialog.	No stock deduction.	Pass
Override text	TC39 D/N: Submit short override reason.	Reason is rejected before stock change.	Pass
Documented override	TC40 D/P/N: Supply a sufficient override reason.	Dispensing succeeds; reason retained in transaction, excluded from general audit.	Pass
Dispensing	TC41 D/P/N: Dispense six units across two batches.	Stock reduced correctly; earliest expiry selected first.	Pass
Insufficient stock	TC42 D: Request more than available quantity.	Rejected atomically; stock unchanged.	Pass
Read isolation	TC43 D: Mutate returned medicine quantity.	Stored stock is unchanged.	Pass
Assistant restriction	TC44 D/P: Assistant restocks or generates report.	Service denies both restricted operations.	Pass

Feature	Test Case	Expected Outcome	Pass/Fail
Doctor/ admin restriction	**TC45 D: Doctor or administrator dispenses medicine.**	**Service denies dispensing.**	**Pass**
Cached inventory	TC46 N: Dispense, then return to cached inventory page.	Table reloads with updated quantity.	Pass
Rollback	TC47 P: Inject test constraint failure after stock update.	Stock and dispensing unit of work rolls back; connection remains usable.	Pass
Closed storage	TC48 P: Register against closed database.	Storage failure is raised; no success result.	Pass
Process lock	TC49 P: Open same database in a second context.	Second opening rejected.	Pass
Notification create	TC50 D/N: Create from existing patient/dispensing transaction.	Message identifies fixture patient and medicine; status PENDING.	Pass
Notification ownership	TC51 P: Link a transaction to another patient.	Rejected.	Pass
Missing transaction	TC52 D: Create notification for unknown transaction.	Rejected.	Pass
Notification status	TC53 D/N: Confirm delivery then mark sent; repeat.	First update records SENT; repeat update rejected.	Pass
Notification snapshots	TC54 D/P: Mutate status on a returned notification.	Stored status is unchanged until authorized service update.	Pass
Notification restart	TC55 P: Restart after marking sent.	ID and SENT status persist.	Pass
Reports	TC56 D/N: Generate selected saved-visit report.	Report contains stored record/ concern and dialog opens/closes.	Pass
Doctor reports	TC57 D: Doctor generates visit report.	Authorized report text includes record ID.	Pass
Paging scale	TC58 U/V: Load 10,000+ fixture records and last page.	At most 25 displayed rows; last/ filter pages have expected values.	Pass
Background loading	TC59 U: Record thread used by page source.	Query runs off Swing EDT.	Pass
Stale results	TC60 U: Let an old request complete after a newer query.	Old result cannot replace current rows.	Pass

Feature	Test Case	Expected Outcome	Pass/Fail
Load failure/ retry	**TC61 U: Force source exception then restore source.**	**Rows/count/selection clear; subsequent reload succeeds.**	**Pass**
Role navigation	TC62 U: Open nurse, assistant, doctor, admin shells.	Allowed screens appear; disallowed role shells fail.	Pass
AI navigation	TC63 U: Nurse opens AI Nurse Assistant.	Assistant page constructs and renders.	Pass
UI resize	TC64 N: Refresh dashboard and resize test window.	Workflow completes and window stays usable in tested dimensions.	Pass
Shutdown	TC65 N: Cancel close, then confirm close.	Cancel retains window; confirmation closes application.	Pass
Restart data	TC66 P: Reopen saved database.	Accounts, campuses, audit, patients, visits and stock checked by suite survive.	Pass
Migration	TC67 P: Re-run interrupted version-1-to-2 migration fixture.	Existing accounts and patients preserved.	Pass
Password storage	TC68 D/P: Inspect audit/repository hash values in test fixture.	Password strings absent from audit and stored hashes.	Pass
AI live generation	TC69: Send non-identifying prompt with valid key/model.	A genuine provider response is displayed; no success established here.	Not Tested
AI missing key	TC70: Clear key in a fresh Java process and Send.	Useful configuration error without provider request. Source inspected; not executed in current run.	Not Tested
AI HTTP failures	TC71: Reproduce 404/429/timeout with current build.	Specific user error; UI re-enables Send. Earlier screenshots showed errors; current scenarios not executed.	Not Tested
AI privacy patterns	TC72: Submit email, phone, labeled student ID.	Reject before cloud send; source rule not exercised here.	Not Tested
AI input boundaries	TC73: Send empty, over-3,500-character and duplicate-pending messages.	Validation prevents invalid or duplicate request; not executed here.	Not Tested
AI long output	TC74: Receive long/multipart response and resize.	Readable wrapped text with correct response extraction; multipart parser limitation unresolved.	Not Tested

Feature	Test Case	Expected Outcome	Pass/Fail
AI lifecycle	**TC75: Start request then log out/ navigate away.**	**Session revocation and late-result handling need additional implementation/review.**	**Not Tested**
Manual accessibility	TC76: Keyboard-only, screen reader, non-default DPI and multiple monitors.	Controls remain accessible and unclipped; no current manual evidence.	Not Tested
Recovery	TC77: Restore closed database and key backup into isolated folder.	Records reopen correctly; complete recovery workflow not exercised.	Not Tested